

# Guía de Laboratorio · Sesión 2

## Análisis de integridad a escala + scoring con IA

**Módulo:** Gestión de Ciber-Recuperación · **Código:** 92-EAN  
**Universidad Ean · Educación Continua · Docente:** Jawy Andrés Romero Pinto  
**Duración:** 60 minutos

### Idea del taller

En la Sesión 1 analizaste un puñado de archivos **a mano**. Un backup real tiene **miles**. Hoy automatizamos las mismas señales forenses para **puntuar el respaldo completo** y saber dónde mirar primero y **cuál copia está limpia** para recuperar.

Dos tiempos: **Parte 1** mide a escala (entropía global y por bloques, magic, similitud); **Parte 2** puntúa con **scikit-learn** y usa el copiloto **local**.

Repositorio del curso: `ciberrecuperacion-ean` (mismo entorno de la Sesión 1).

## Mapa de la hora

| Tiempo    | Bloque                      | Qué hacemos                                                                   |
|-----------|-----------------------------|-------------------------------------------------------------------------------|
| 00-08 min | Setup                       | Entorno del kit, <code>make check</code> en verde                             |
| 08-38 min | Parte 1 · Análisis a escala | Cifrado parcial, <code>extraer_features.py</code> → <code>features.csv</code> |
| 38-55 min | Parte 2 · Scoring + IA      | <code>score.py</code> (IsolationForest) + copiloto local                      |
| 55-60 min | Cierre                      | ML + reglas + humano; por qué local                                           |

## Setup (8 min)

```
cd ciberrecuperacion-ean/core
git pull # trae la carpeta 02 de esta sesión
bash setup.sh --local # o --cloud (Azure/Groq). Instala scikit-learn en el venv
make check # debe dar [OK]
```

✓ **Checkpoint setup:** `make check` responde `[OK]` y el venv está activo.

## Parte 1 · Análisis de integridad a escala (30 min)

Guía completa: `02-analisis-de-integridad-datos/parte-1-analisis-a-escala.md`.

**Genera el backup del caso (a escala):**

```
cd ../02-analisis-de-integridad-datos
python3 scripts/generar_dataset.py
cd datasets/backup_caso2
ls -R | wc -l # demasiados para mirar uno por uno
```

### 1. Confirma el cifrado parcial a mano (cabecera legible + cola aleatoria):

```
head -c 120 manual_parcial.pdf # texto/PDF legible
tail -c 120 manual_parcial.pdf | od -c # bytes aleatorios (cifrado) al final
```

La entropía **global** no basta: hay que medirla **por bloques**.

### 2. Extrae las señales de TODO el backup:

```
python3 ../../scripts/extraer_features.py . -o features.csv
column -s, -t features.csv | sort -k3 -r | head -12 # ordena por entropía
```

3. Interpreta (como investigador):

- `entropy` ~8 y `chunk_ent_std` ~0 → **cifrado total** ( `.locked3d` , `.crypt` ).
- `chunk_ent_std` alto → **cifrado parcial** ( `*_parcial.*` ).
- `magic_mismatch` = 1 → **extensión falsa** ( `factura_2024.pdf` es PNG... ).
- `max_similarity` alto entre varios → **variantes** casi duplicadas ( `facturas/*` ).

El **agente de Warp (OZ)** te ayuda con los comandos ( `column` , `sort` , `od` ); la lectura de las señales la haces tú.

✓ **Checkpoint Parte 1:** tienes `features.csv` y sabes leer, por columnas, qué está cifrado (total/parcial), qué tiene extensión falsa y qué son variantes.

Parte 2 · Scoring con scikit-learn + IA (17 min)

Guía: `02-analisis-de-integridad-datos/parte-2-scoring-ia.md`.

1. Puntúa el backup (IsolationForest + reglas):

```
source ../../core/.venv/bin/activate # trae scikit-learn
python3 ../../scripts/score.py features.csv --top 16
```

Salida: `corrupcion_score` (0-100), bandera de **anomalía** (ML) y `clasificacion` por reglas. Se guarda en `scores.csv`.

2. Lee con criterio (tres capas que se complementan):

1. **ML** (score): dónde mirar primero entre miles.
2. **Reglas** (clasificación): qué tipo de problema es.
3. **Tú**: confirmas y decides la copia a recuperar.

Discute: ¿dónde coinciden ML y reglas?, ¿algún **falso positivo** (duplicado legítimo con score alto)?, ¿qué archivos `limpio` sirven para recuperar?

3. Copiloto de IA local sobre el resultado:

```
ollama run llama3.1:8b "Eres analista de ciber-recuperacion. Con esta tabla de scoring de un backup, dime: (1) que priorizar
```

( `/bye` para salir.) Contrasta con **tu** lectura: ¿coincide?, ¿se le escapó el cifrado parcial?, ¿aluciné algún archivo?

En setup **hosted** (Azure/Groq): aquí el dato es **sintético**. Con evidencia real, el modelo va **local** (regla datos→backend).

✓ **Checkpoint Parte 2:** generaste `scores.csv`, interpretaste ML + reglas y corriste el copiloto **local** contrastándolo con tu análisis.

Cierre (5 min)

La regla de oro de hoy: **el ML prioriza, las reglas clasifican, el analista decide** — ninguna capa basta sola. Así pasamos de miles de archivos a una **lista priorizada** para hallar lo corrupto y **la copia limpia**: la base de la recuperación con integridad (Sesiones 4 y 5).

Qué alistar para la Sesión 3

El lab de hoy funcionando ( `features.csv` y `scores.csv` generados), el entorno con scikit-learn y el repo actualizado ( `git pull` ). La Sesión 3 (reglas **YARA/Sigma** generadas con IA) construye sobre este entorno.

Referencia rápida de comandos

```
python3 scripts/generar_dataset.py # backup del caso (a escala)
head -c 120 manual_parcial.pdf # cabecera legible...
tail -c 120 manual_parcial.pdf | od -c # ...cola cifrada (parcial)
```

```
python3 ../../scripts/extraer_features.py . -o features.csv
source ../../core/.venv/bin/activate
python3 ../../scripts/score.py features.csv --top 16
ollama run llama3.1:8b "...: $(cat scores.csv)" # copiloto local
```